Основы Swift

Синтаксис, типы данных и управление выполнением

- Чтение и запуск консольных программ
- Работа со значениями и базовыми типами
- Условия, циклы, коллекции и функции

Команда 2 :

Власенко Виталия

Рзаева Элеонора

Шевченко Дарья

Как появился Swift

- **2010:** Крис Латтнер начинает разработку языка в Apple
- **2014:** Apple представляет Swift на WWDC
- **Цель:** безопаснее и понятнее код без потери производительности
- **2015:** исходный код открыт; Swift развивается и на Linux
- **2024:** Swift 6 усиливает проверку конкурентного доступа

Особенности: строгие типы, `Optional`, совместимость с Objective-C

Swift 6 и Swift 6.4.0: в чём разница

Компилятор проверяет Swift-код и создаёт исполняемую программу.

Swift 6	Swift 6.4.0
Правила проверки исходного кода	Выпуск установленного компилятора
`-swift-version 6`	`swift --version`

В этом курсе: компилятор 6.4.0, правила Swift 6.

Документация: `docs.swift.org` и `swift.org`

Как запускать Swift

- Локально: Swiftly или Xcode
- Docker: официальный образ `swift:6.4.0-noble`
- Онлайн: SwiftFiddle или OneCompiler
- Проверка версии: `swift --version`

Подробные команды и ограничения: `README.md`

Консольная программа

- Инструкции на верхнем уровне файла — точка входа
- `print` выводит данные в стандартный поток вывода
- Отдельная функция `main()` не обязательна

```
print("Программа завершена")
```

Файл: `code_1/first_program.swift`
Запуск: `swift -swift-version 6 code_1/first_program.swift`

Константы и переменные

- `let` — однократное присваивание
- `var` — возможность изменения значения
- Вывод типа из инициализатора; явный тип после `:`

```swift
let courseTitle = "Основы Swift"
var completedLessons = 0
completedLessons += 1
```

Файл: `code_1/variables.swift`
Запуск: `swift -swift-version 6 code_1/variables.swift`

Основные типы Swift

- `Int`, `Double`, `Float` — числовые значения
- `Bool` — логическое значение
- `Character` — символ; `String` — строка

```swift
let itemCount: Int = 8
let averageScore: Double = 4.5
let grade: Character = "A"
let subject: String = "Swift"
```

Файл: `code_1/basic_types.swift`
Запуск: `swift -swift-version 6 code_1/basic_types.swift`

Операторы и выражения

- `+`, `-`, `*`, `/`, `%` — арифметика
- `==`, `<`, `>=` — сравнение; результат `Bool`
- `&&`, `||` — объединение условий
- Деление значений `Int` даёт целое число

```swift
let completedTasks = 7
let totalTasks = 10
let completionPercent = completedTasks * 100 / totalTasks
let shouldContinue = completedTasks < totalTasks && completionPercent >= 50
print("Выполнено: \(completionPercent)%")
```

Файл: `code_1/operators.swift`
Запуск: `swift -swift-version 6 code_1/operators.swift`

Строки и Unicode

- `String` хранит текст Unicode
- `+` соединяет строки; `\(...)` подставляет значение
- `count` считает символы, а не байты
- Индексы строки имеют тип `String.Index`

```swift
let topic = "Swift"
let headline = "Курс: " + topic
let report = "\(headline), символов: \(headline.count)"
let accentedWord = "cafe\u{301}"                    // café
print("Символов: \(accentedWord.count)")            // 4
print("Байт UTF-8: \(accentedWord.utf8.count)")     // 6
let initialCharacter = accentedWord[accentedWord.startIndex]
print("Первый символ: \(initialCharacter)")
```

Файл: `code_1/strings.swift`
Запуск: `swift -swift-version 6 code_1/strings.swift`

Преобразование типов

- Swift не смешивает числовые типы автоматически
- `Double(value)` — явная конвертация
- `Int(text)` возвращает `Int?`: преобразование может не удаться

```swift
let count = 5
let average = Double(count) / 2
let source = "42"
if let parsed = Int(source) {            // выведет "43"
    print(parsed + 1)
}
let invalidSource = "42a"
if let parsed = Int(invalidSource) {     // выведет "Некорректный ввод: 42a"
    print("Получено: \(parsed)")
} else {
    print("Некорректный ввод: \(invalidSource)")
}
```

Файл: `code_1/conversions.swift`
Запуск: `swift -swift-version 6 code_1/conversions.swift`

Блоки и области видимости

- `{ ... }` ограничивает область действия объявлений
- Внутренний блок видит внешние имена
- Локальное объявление может скрывать внешнее

```swift
let mode = "обычный"
do {
    let mode = "тестовый"
    print(mode)
}
print(mode)
```

Файл: `code_1/scopes.swift`
Запуск: `swift -swift-version 6 code_1/scopes.swift`

Опциональные значения

- `Int?` — число либо отсутствие значения `nil`
- `if let` проверяет наличие значения и извлекает его
- `guard let` допускает ранний выход из области

```swift
let rawQuantity = "15"
let quantity: Int? = Int(rawQuantity)
if let quantity {
    print("Количество: \(quantity)")
} else {
    print("Некорректное число")
}
func printQuantity(from source: String) {
    guard let parsedQuantity = Int(source) else {
        print("Не удалось прочитать количество: \(source)")
        return
    }
    print("Прочитано: \(parsedQuantity)")
}
```

Файл: `code_1/optionals.swift`
Запуск: `swift -swift-version 6 code_1/optionals.swift`

Условия: `if`, `else`, `guard`

- Условие имеет тип `Bool`
- `else if` проверяется после неудачного `if`
- `guard` требует выполнения условия для продолжения

```swift
if score >= 80 {
    print("Высокий результат")
} else if score >= 50 {
    print("Зачёт")
} else {
    print("Повторить тему")
}
func accessDecision(for age: Int, hasTicket: Bool) -> String {
    guard age >= 16 else {
        return "Доступ запрещён по возрасту"
    }
    ...
}
```

Файл: `code_1/conditions.swift`
Запуск: `swift -swift-version 6 code_1/conditions.swift`

`switch`: выбор ветви

- `case` задаёт значение или образец для сопоставления
- `default` обрабатывает остальные случаи
- Все возможные значения должны быть учтены
- Перехода к следующей ветви по умолчанию нет

```swift
switch statusCode {
case 200...299: print("Успех")
case 404: print("Не найдено")
default: print("Другой статус")
}
```

Файл: `code_1/switches.swift`
Запуск: `swift -swift-version 6 code_1/switches.swift`

Циклы и диапазоны

- `for-in` — перебор значений последовательности
- `1...3` включает обе границы; `0..<3` исключает верхнюю
- `while` проверяет условие до прохода
- `repeat-while` выполняет блок хотя бы один раз

```swift
for lessonNumber in 1...3 {
    print("Занятие: \(lessonNumber)")
}

while remainingAttempts > 0 {
    ...
}

repeat {
    ...
} while checkCount < 2
```

Файл: `code_1/loops.swift`
Запуск: `swift -swift-version 6 code_1/loops.swift`

Последовательности и итераторы

- `Sequence` — источник элементов для перебора
- `makeIterator()` создаёт итератор
- `next()` возвращает следующий элемент или `nil`
- `for-in` выполняет этот перебор автоматически

```swift
let chapters = ["Типы", "Условия", "Циклы"]
var iterator = chapters.makeIterator()
while let chapter = iterator.next() {
    print("Следующий: \(chapter)")
}
```

Файл: `code_1/iterators.swift`
Запуск: `swift -swift-version 6 code_1/iterators.swift`

Массивы `Array`

- Упорядоченное хранение элементов одного типа
- `append` добавляет элемент; `count` возвращает количество элементов массива.
- Индекс проверяется через `indices.contains(...)`
- `first` возвращает первый элемент или nil, если массив пуст.

```swift
var topics = ["Типы", "Условия"]
topics.append("Циклы")
let index = 2
if topics.indices.contains(index) {
    print(topics[index])
}
if let firstTopic = topics.first {
    print("Первый раздел: \(firstTopic)")
}
for (offset, topic) in topics.enumerated() {
    print("\(offset): \(topic)")
}
```

Файл: `code_1/arrays.swift`
Запуск: `swift -swift-version 6 code_1/arrays.swift`

Множества и словари

- `Set` содержит уникальные элементы
- `Dictionary` связывает ключи со значениями
- Поиск по ключу возвращает Optional
- Порядок перебора элементов не гарантирован

```swift
let uniqueTopics: Set<String> = ["Типы", "Типы", "Циклы"]
let scores = ["Типы": 90, "Циклы": 85]
print(uniqueTopics.contains("Циклы"))
if let score = scores["Типы"] {
    print(score)
}

for topic in scores.keys.sorted() {    // вывод ключей в алфавитном порядке
    if let score = scores[topic] {
        print("\(topic): \(score)")
    }
}
```

Файл: `code_1/sets_dictionaries.swift`
Запуск: `swift -swift-version 6 code_1/sets_dictionaries.swift`

Функции: параметры и результат

- `func` объявляет функцию
- Параметры имеют имена и типы
- `->` указывает тип результата; `return` возвращает его
- Метки аргументов делают вызов читаемым

```swift
func progressMessage(for completed: Int, outOf total: Int) -> String {
    return "\(completed) из \(total)"
}
print(progressMessage(for: 3, outOf: 5))
```

Файл: `code_1/functions.swift`
Запуск: `swift -swift-version 6 code_1/functions.swift`

Консольный ввод-вывод

- `print` выводит текст
- `readLine()` читает строку или возвращает `nil`
- `CommandLine.arguments` содержит параметры запуска
- Текстовый ввод проверяется перед вычислениями

```swift
let section = CommandLine.arguments.dropFirst().first ?? "Основы"
print("Раздел: \(section)")
if let rawCount = readLine(), let count = Int(rawCount), count >= 0 {
    print("Задач: \(count)")
} else {
    print("Нет корректного неотрицательного числа")
}
```

Файл: `code_1/console_io.swift`
Запуск: `printf '3\n' | swift -swift-version 6 code_1/console_io.swift "Коллекции"`